PCCCS495: Advanced Programming (OOP) Lab

Term-II Individual Project (Spring 2026)

Smart Elevator Control System

Name: Sandipta Bhattacharyya

Roll No.: 04

Instructors: Dr. Swagatam Basu, Dr. Susovan Jana, Dr. Soumadip Biswas

Spring 2026

Problem Statement:

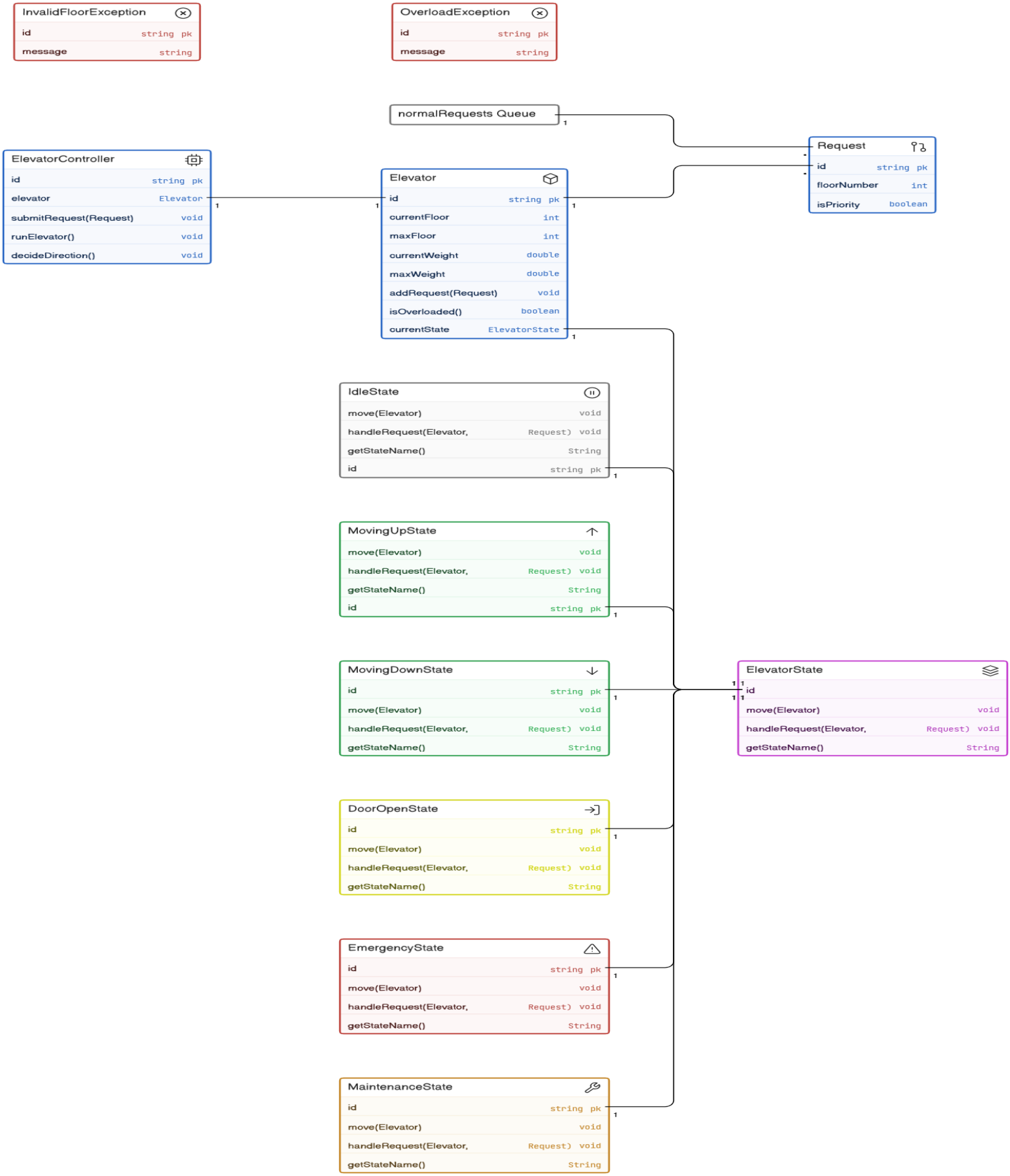
Modern elevators operate under multiple dynamic conditions such as movement direction, door operation, emergency handling, and maintenance mode. Traditional implementations rely heavily on conditional logic, making the system difficult to maintain and extend.

This project aims to design a Smart Elevator Control System using object-oriented principles and the State Design Pattern. The system models elevator behavior through distinct states such as Idle, Moving Up, Moving Down, Door Open, Emergency, and Maintenance.

The system supports priority requests, overload detection, and safety mechanisms, ensuring realistic simulation. The objective is to demonstrate clean architecture, modularity, extensibility, and proper use of OOP concepts rather than building a large-scale system.

System Design:

1. UML Diagram



2. Key classes:

Elevator

- Stores elevator state, floor, weight, and request queues.
- Acts as the core model.

ElevatorController

- Controls elevator logic and transitions.
- Acts as the brain of the system.

ElevatorState (Interface)

- Defines behavior contract for all states.

State Classes

- Each state defines different behavior:
    - IdleState
    - MovingUpState
    - MovingDownState
    - DoorOpenState
    - EmergencyState
    - MaintenanceState

ConsoleUI

- Handles user interaction.

3. Package Structure:

model → Elevator, Request

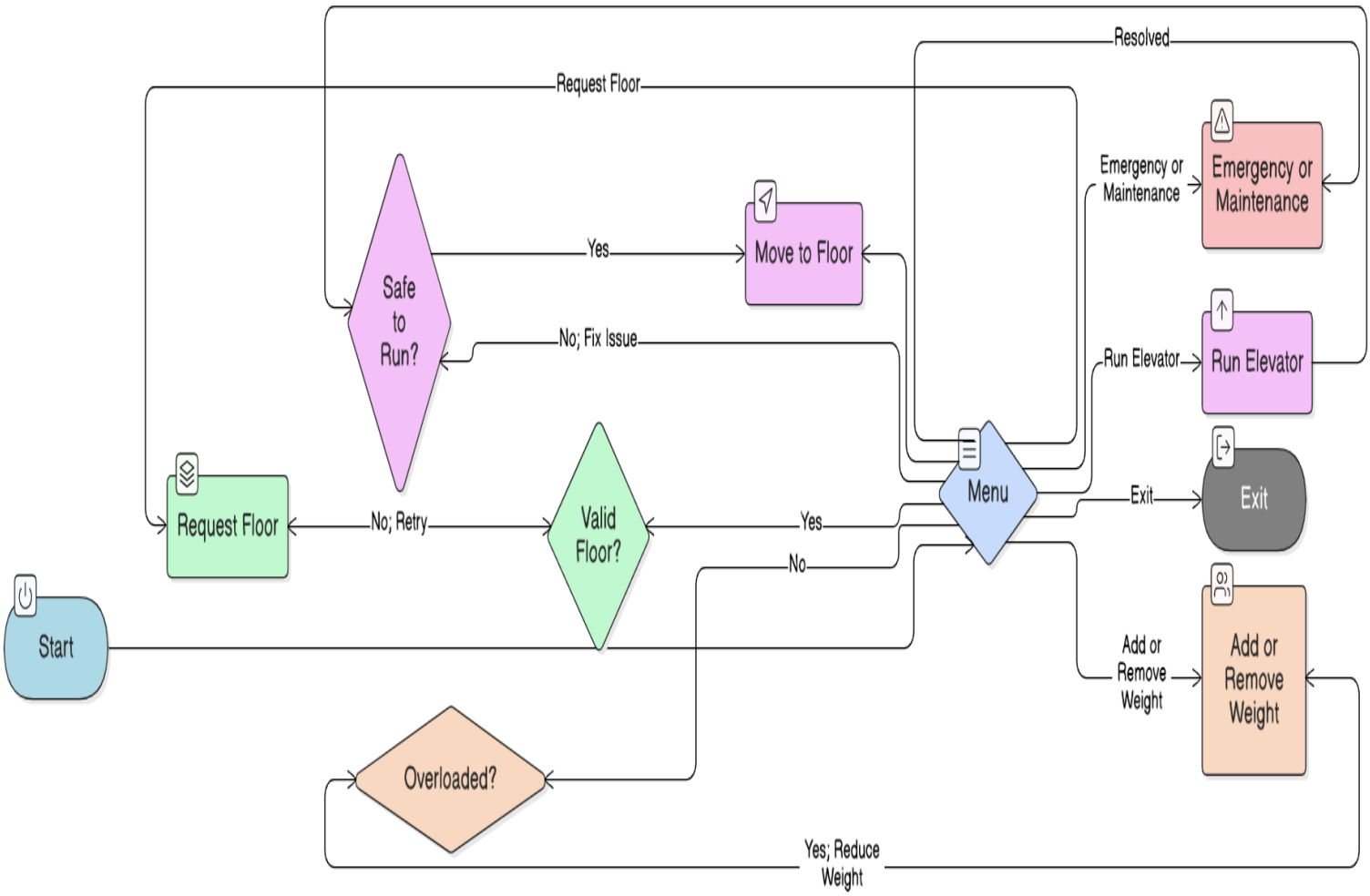
state → All state classes

controller → ElevatorController

ui → ConsoleUI

exception → Custom exceptions

4. Interaction Flow



- User interacts with the system through the **Console UI menu**
- User selects an option (request floor, add/remove weight, run elevator, etc.)
- Input is **validated** (e.g., valid floor range, valid weight)
- Valid requests are sent to the **ElevatorController**
- Controller forwards the request to the **current state of the elevator**
- Elevator stores requests in **normal or priority queues**
- When “Run Elevator” is selected:
  - System checks **overload condition**
  - Checks **emergency/maintenance state**
- If safe, the controller calls **move()** on the current state
- Elevator moves floor by floor and updates state dynamically
- After completing requests, elevator returns to **Idle state**
- System loops back to the **menu for next user action**

OOPS CONCEPT DESIGN:

1. Abstraction

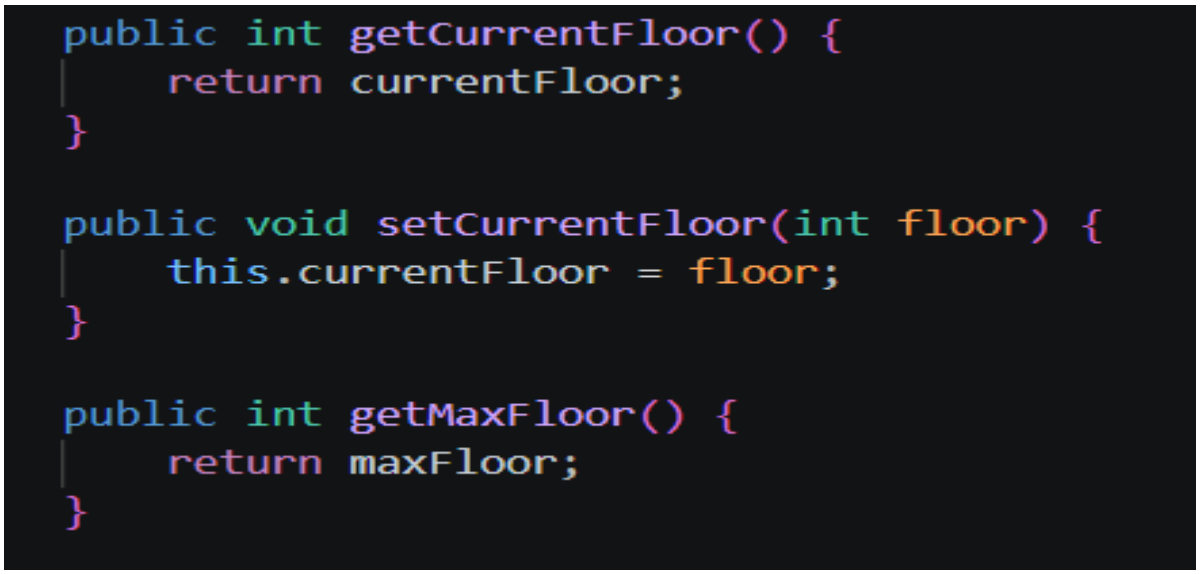
Abstraction is implemented using the **ElevatorState interface**, which defines common behavior for all elevator states.

```java
package state;
import model.Elevator;
import model.Request;
public interface ElevatorState {

    void handleRequest(Elevator elevator, Request request);

    void move(Elevator elevator);

    String getStateName();
}
```

Explanation: The interface defines *what* actions a state must perform without specifying *how*. Each concrete state (like MovingUpState, IdleState) provides its own implementation, allowing flexible behavior design.

2. Encapsulation

Encapsulation is applied in the **Elevator class**, where data is kept private and accessed through methods



```java
public int getCurrentFloor() {
    return currentFloor;
}

public void setCurrentFloor(int floor) {
    this.currentFloor = floor;
}

public int getMaxFloor() {
    return maxFloor;
}
```

Explanation: Sensitive data such as floor and weight are protected using private access. This prevents direct modification and ensures controlled updates through setter methods.

3. Inheritance

All state classes inherit behavior from the **ElevatorState interface**

```
public class IdleState implements ElevatorState {
```

Explanation: By implementing the interface, each state class inherits the contract of required methods. This promotes code reuse and consistency across different states.

4. Polymophism

Polymorphism is achieved when the system calls methods on the current state dynamically.

```
    elevator.getCurrentState().move(elevator);
```

Explanation: The same method move() behaves differently depending on the current state (e.g., Idle, MovingUp, Emergency). This enables dynamic behavior without complex conditional statements.

5. Exception Handling

Custom exceptions are used to handle invalid inputs and unsafe conditions.

```
case 1:
    try {
        System.out.print(s: "Enter floor number: ");
        int floor = scanner.nextInt();

        if (floor < 1 || floor > elevator.getMaxFloor()) {
            throw new RuntimeException(message: "Invalid floor selected!");
        }

        controller.submitRequest(new Request(floor, priority: false));

    } catch (Exception e) {
        System.out.println(e.getMessage());
    }
```

Explanation: Exception handling ensures that invalid operations are detected and handled gracefully, preventing system crashes and improving reliability.

6. Collections

Queues are used to manage elevator requests efficiently.

```
private Queue<Request> normalRequests;
private Queue<Request> priorityRequests;
```

Explanation: Queues maintain the order of requests and allow efficient processing. Separate queues for priority and normal requests ensure correct scheduling.

7. Thread

Threading is used to simulate real-time elevator movement.

```
try {
    Thread.sleep(millis: 1000);
} catch (InterruptedException e) {
    e.printStackTrace();
}
```

Explanation: The delay simulates real-world movement between floors, making the system behavior more realistic.

8. Design Pattern

The project uses the **State Design Pattern** to manage elevator behavior

```
elevator.setCurrentState(new DoorOpenState());
```

Explanation: The State Pattern allows the elevator to change its behavior dynamically based on its current state, eliminating complex conditional logic and improving scalability.

Implementation Highlights:

1. State Transition Logic

```
if (target > elevator.getCurrentFloor()) {
    elevator.setCurrentState(new MovingUpState());
} else {
    elevator.setCurrentState(new MovingDownState());
}
```

Explanation: This snippet demonstrates how the elevator dynamically changes its behavior based on the requested floor. Instead of using conditional logic throughout the program, the system switches between different state objects. This improves modularity and makes the system easier to extend with new behaviors.

2. Priority Request Handling

```
public void addRequest(Request request)
    if (request.isPriority()) {
        priorityRequests.add(request);
    } else {
        normalRequests.add(request);
    }
}
```

Explanation: This implementation separates priority and normal requests using two queues. It ensures that urgent or VIP requests are handled before regular ones. This improves scheduling efficiency and reflects real-world elevator behavior.

3. Overload Detection and Safety Detection

```
catch (OverloadException e) {
  System.out.println(e.getMessage());
  System.out.println(x: "Please reduce weight and try again.");
  elevator.setCurrentState(new IdleState());
```

Explanation: This snippet ensures that the elevator does not operate under unsafe conditions. When the weight exceeds the limit, the system blocks movement and resets the state to Idle, allowing the user to correct the issue. This demonstrates safety-first design.

4. Movement Simulation Using Thread delay

```
try {
    Thread.sleep(millis: 1000);
} catch (InterruptedException e) {
    e.printStackTrace();
}
```

Explanation: This snippet simulates real-world elevator movement by introducing a delay between floor transitions. It enhances the realism of the system and demonstrates basic use of threading concepts.

Test Cases and Error Handling

Test Cases:

1. Valid Floor Request

- **Input:** Floor = 5
- **Expected Output:** Elevator moves to floor 5

- **Result:** Passed

Verifies basic functionality.

2. Priority Floor Request

- **Input:** Priority Floor = 9
- **Expected Output:** Priority request is served before normal requests
- **Result:** Passed

Verifies priority queue handling.

3. Invalid Floor Request

- **Input:** Floor = -1 or Floor > maxFloor
- **Expected Output:** Error message displayed
- **Result:** Passed

Verifies input validation and exception handling.

4. Overload Condition

- **Input:** Weight exceeds maximum limit
- **Expected Output:** Warning displayed and elevator movement blocked
- **Result:** Passed

Verifies safety constraint.

5. Emergency Mode

- **Input:** Activate emergency mode
- **Expected Output:** Elevator stops and cannot move
- **Result:** Passed

Verifies emergency handling.

6. Maintenance Mode

- **Input:** Activate maintenance mode
- **Expected Output:** Elevator becomes unavailable for operation
- **Result:** Passed

Verifies restricted operation.

7. Weight Removal

- **Input:** Remove weight after overload
- **Expected Output:** Elevator resumes normal operation
- **Result:** Passed

Verifies recovery from error state.

Edge Cases Handled:

- Floor number below 1
- Floor number exceeding maximum limit
- Removing more weight than current weight
- Running elevator in overloaded condition
- Running elevator during emergency or maintenance mode

These ensure robustness of the system.

Error Handling Mechanisms:

The system uses structured error handling techniques to prevent failures and ensure smooth execution.

1. Custom Exception Handling

```
try {
    System.out.print(s: "Enter floor number: ");
    int floor = scanner.nextInt();

    if (floor < 1 || floor > elevator.getMaxFloor()) {
        throw new RuntimeException(message: "Invalid floor selected!");
    }

    controller.submitRequest(new Request(floor, priority: false));

} catch (Exception e) {
    System.out.println(e.getMessage());
```

Explanation: Handles invalid user input explicitly.

2. Overload Protection

```java
if (elevator.isOverloaded()) {
    System.out.println(x: "Elevator overloaded! Please remove weight before running.");
    break;
}
```

Explanation: Prevents unsafe operation.

3.General Exception Handling

```java
catch (Exception e) {
    System.out.println(e.getMessage());
```

Explanation: Prevents program crash due to unexpected errors.

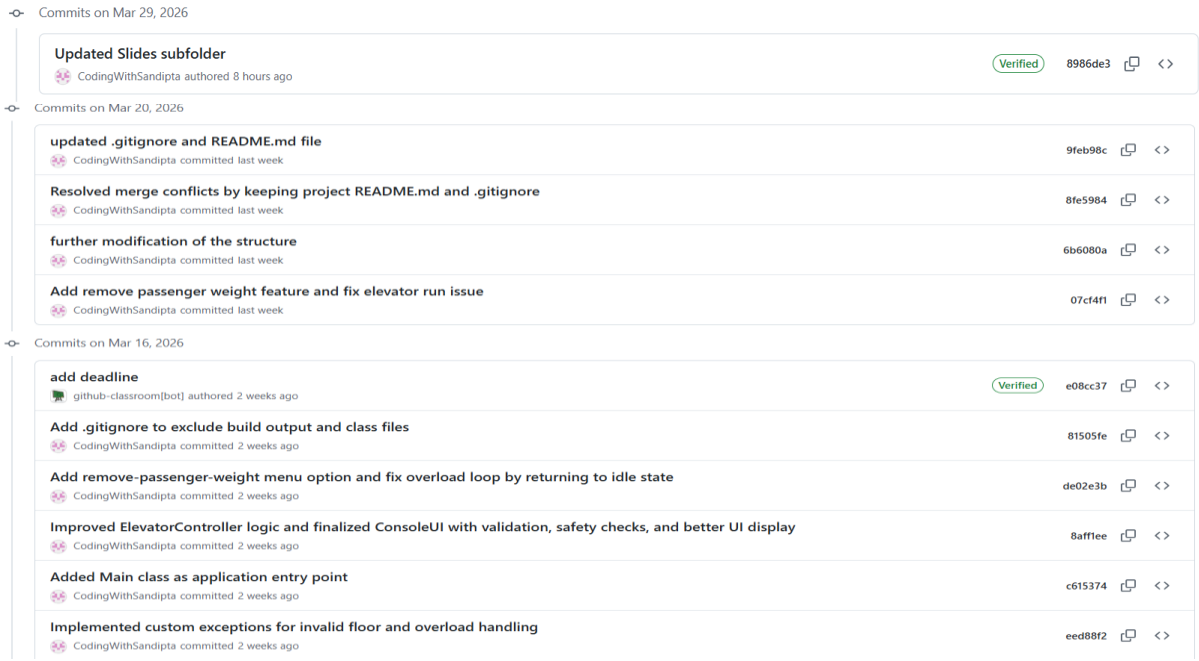
Failure Scenarios:

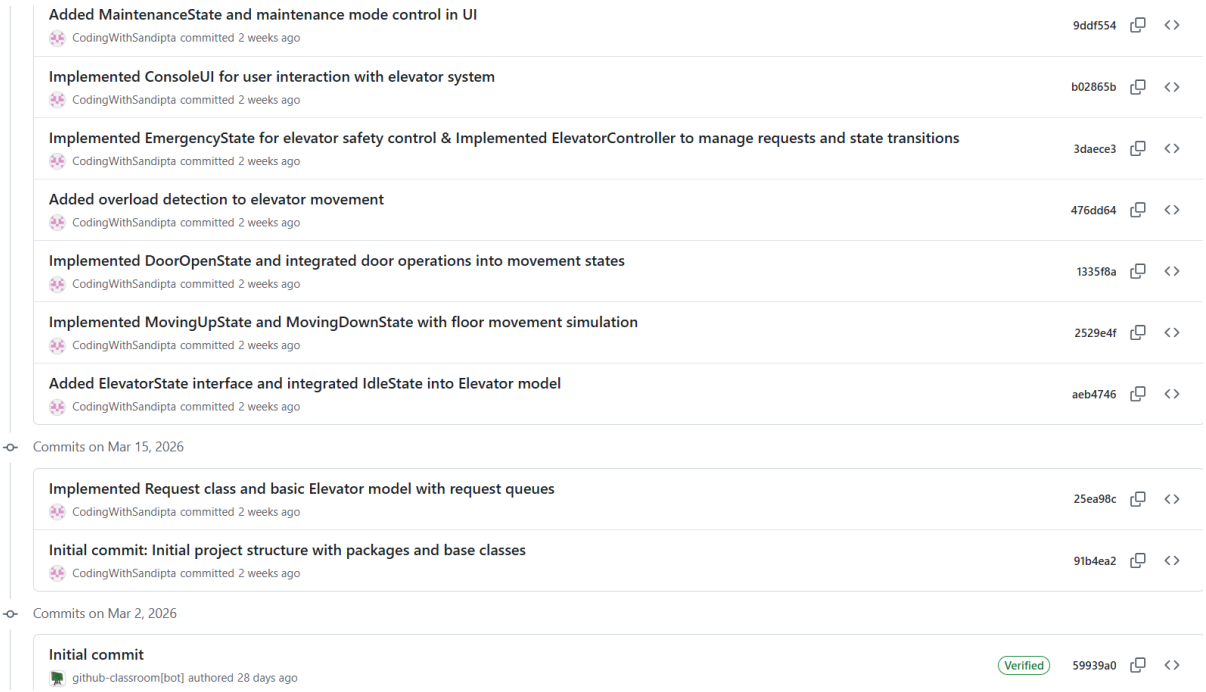
- Invalid user input → handled using exceptions
- Overload condition → movement blocked
- Emergency activation → system restricted
- No requests → system returns to Idle state

System remains stable in all scenarios.

Git WorkFlow:

1. Commit History





A total of **20+ commits** were made during the development process. Each commit represents a specific feature or improvement.

The project was developed using **Git for version control**, following an incremental and modular development approach. Instead of uploading the entire project at once, the system was built step by step with regular commits representing meaningful progress.

Each component such as model classes, state implementations, controller logic, and user interface was developed and committed separately. This ensured a clear development history and made debugging and improvements easier.

Conclusion

The Smart Elevator Control System successfully demonstrates the application of object-oriented programming principles and design patterns in solving a real-world problem. The system models elevator behavior using the State Design Pattern, allowing dynamic transitions between different operational states such as Idle, Moving Up, Moving Down, Emergency, and Maintenance.

Key features such as priority request handling, overload detection, and safety restrictions were implemented to ensure realistic and robust system behavior. The use of abstraction, encapsulation, inheritance, and polymorphism helped in designing a modular and maintainable architecture.

Additionally, the separation of concerns between user interface, controller logic, and system model improved code clarity and extensibility. Overall, the project achieves its objective of demonstrating disciplined object-oriented design rather than focusing on excessive features.

Future Scope

The current system can be further enhanced with the following improvements:

- **Graphical User Interface (GUI):** Replace the console interface with a Java Swing or JavaFX-based UI for better usability
- **Multiple Elevator Support:** Extend the system to handle multiple elevators with intelligent scheduling algorithms
- **Advanced Scheduling Algorithms:** Implement optimized algorithms to minimize waiting time and improve efficiency
- **Real-Time Sensor Simulation:** Integrate sensors for door status, weight detection, and floor detection for more realistic simulation
- **Multithreading Support:** Allow multiple user requests simultaneously using proper concurrency handling
- **Logging System:** Maintain logs of elevator activity for monitoring and debugging purposes

These enhancements would make the system closer to a real-world elevator control system and further strengthen its design and functionality.

Appendix:

Copy of Project Proposal

Project Title

Smart Elevator Control System using State Pattern with Priority and Safety Management

Problem Statement

Modern elevators must handle multiple floor requests efficiently while ensuring safety and proper behavior under varying operational conditions. Traditional implementations often rely on complex conditional logic, which becomes difficult to maintain and extend when additional behaviors such as emergency mode, maintenance mode, priority servicing, or overload protection are introduced. This project aims to design and simulate a Smart Elevator Control System that models real-world elevator behavior using a structured object-oriented approach. The system will demonstrate how different operational states (Idle, Moving Up, Moving Down, Door Open, Emergency, Maintenance) can be managed using the State Design Pattern to eliminate excessive conditional branching. The project emphasizes clean architecture, separation of concerns, and extensibility, ensuring that new states or control rules can be added without modifying core logic.

Target User

- Students studying object-oriented design concepts
- Developers learning design patterns
- Academic demonstration of real-world system modeling

Core Features

- Simulate a single elevator in a 10-floor building
- Handle multiple floor requests using a queue
- Separate priority request handling
- Direction-based request servicing (up/down logic)
- State-based behavior management (Idle, Moving, Door Open, etc.)
- Emergency stop mode
- Maintenance mode (disables normal operation)
- Overload detection with movement restriction
- Custom exception handling for invalid operations
- Console-based interactive interface

OOP Concepts to be Used

Abstraction

- ElevatorState interface defining common behavior for all states.

Inheritance

- Concrete state classes (IdleState, MovingUpState, EmergencyState, etc.) implementing the abstract state behavior.

Polymorphism

- Different states overriding behavior methods dynamically at runtime.

Exception Handling

- Custom exceptions such as OverloadException, InvalidFloorException, and EmergencyActiveException.

Collections

- Queue / PriorityQueue to manage floor requests efficiently.

Proposed Architecture Description

The system follows a layered object-oriented architecture separating UI, control logic, state management, and domain models. The Elevator class acts as the core entity and maintains its current operational state. Behavior is delegated to concrete state classes implementing the ElevatorState interface, following the State Design Pattern to eliminate complex conditional logic. An ElevatorController coordinates request handling and state transitions. Request queues are managed using Java Collections. Custom exceptions enforce safety constraints such as overload or invalid floor requests. This design ensures modularity, extensibility, and clear separation of concerns.

Copy of Review Mail received:

PCCCS495 Proposal Review – APPROVED_MINOR

From: soumadip.biswas@ieminternational.ai
To: Sandipta Bhattacharyya Sandipta.Bhattacharyya2024@iem.edu.in
Date: Mon, 9 Mar 2026, 1:39 PM

Dear Sandipta Bhattacharryya,

Project: Smart Elevator Control System using State Pattern with Priority and Safety Management
Decision: APPROVED_MINOR
Score: 92

Strengths:
This is an exceptionally strong proposal for a university Java OOP project. The problem statement is clear, concise, and effectively highlights the common pitfalls of traditional imperative approaches, providing a strong justification for the chosen design pattern. The application of the State Design Pattern to manage elevator behavior is central, well-articulated, and an excellent pedagogical choice for demonstrating OOP principles in a practical context. The feature set is comprehensive, covering core functionality (request handling, direction logic) as well as critical safety and operational modes (emergency, maintenance, overload, priority). The proposed layered architecture ensures strong separation of concerns, modularity, and extensibility, which are key learning objectives for OOP. The explicit mention of fundamental OOP concepts and good practices like exception handling further solidifies the proposal's academic value. The scope is well-balanced, providing sufficient complexity without being overwhelming for a student project.

Improvements:
While the proposal is robust, a few minor enhancements could further strengthen it:
1. **Simulation Realism:** For a 'Smart Elevator Control System,' considering a basic time-based simulation (e.g., using `Thread.sleep` or a simple timer for movement and door operations) could enhance realism beyond sequential console input and subtly introduce event-driven concepts.
2. **Testing Strategy:** Briefly mentioning a testing strategy (e.g., unit tests for state transitions, request handling, and error conditions) would demonstrate a more complete software development lifecycle approach.
3. **User Interaction Details:** A very brief expansion on how users would interact with the console interface (e.g., 'users can input floor requests and trigger emergency/maintenance modes via menu options') could add a touch more clarity.

Regards,
Course Instructor